

Compiler

In computing, a **compiler** is software that translates computer code written in one programming language (the *source* language) into another language (the *target* language). The name "compiler" is primarily used for programs that translate source code from a high-level programming language to a low-level programming language (e.g., assembly language, object code, or machine code) to create an executable program.^{[1][2]:p1[3]}

There are many different types of compilers which produce output in different useful forms. A cross-compiler produces code for a different central processing unit (CPU) or operating system than the one on which the cross-compiler itself runs. A bootstrap compiler is often a temporary compiler, used for compiling a more permanent or better optimized compiler for a language.

Related software include decompilers, programs that translate from low-level languages to higher-level ones; programs that translate between high-level languages, usually called source-to-source compilers or transpilers; language rewriters, usually programs that translate the form of expressions without a change of language; and compiler-compilers, compilers that produce compilers (or parts of them), often in a generic and reusable way so as to be able to produce many differing compilers.

A compiler is likely to perform some or all of the following operations, often called phases: preprocessing, lexical analysis, parsing, semantic analysis (syntax-directed translation), conversion of input programs to an intermediate representation, code optimization and machine specific code generation. Compilers generally implement these phases as modular components, promoting efficient design and correctness of transformations of source input to target output. Program faults caused by incorrect compiler behavior can be very difficult to track down and work around; therefore, compiler implementers invest significant effort to ensure compiler correctness.^[4]

Comparison with interpreter

With respect to making source code runnable, an interpreter provides a similar function as a compiler, but via a different mechanism. An interpreter executes code without converting it to machine code.^{[2]:p2} Therefore, some interpreters execute source code while others execute an intermediate form such as bytecode.

A program compiled to native code tends to run faster than when interpreted, while environments with a bytecode-intermediate-form tends toward intermediate-speed. Alternatively, just-in-time compilation allows for native execution speed with a one-time startup processing time cost.

Low-level programming languages, such as assembly and C, are typically compiled, especially when speed is a significant concern, rather than having cross-platform support. For such languages, there are more one-to-one correspondences between the source code and the resulting machine code, making it easier for programmers to control the use of hardware.

In theory, any programming language can be used via either a compiler or an interpreter, but in practice, languages tend to be used with only one or the other. Nonetheless, it is possible to write a compiler for a language that is commonly interpreted. For example, Common Lisp can be compiled to Java bytecode (and then interpreted by the Java virtual machine), to C code (which can then be compiled again to native machine code), or directly to native code.

History

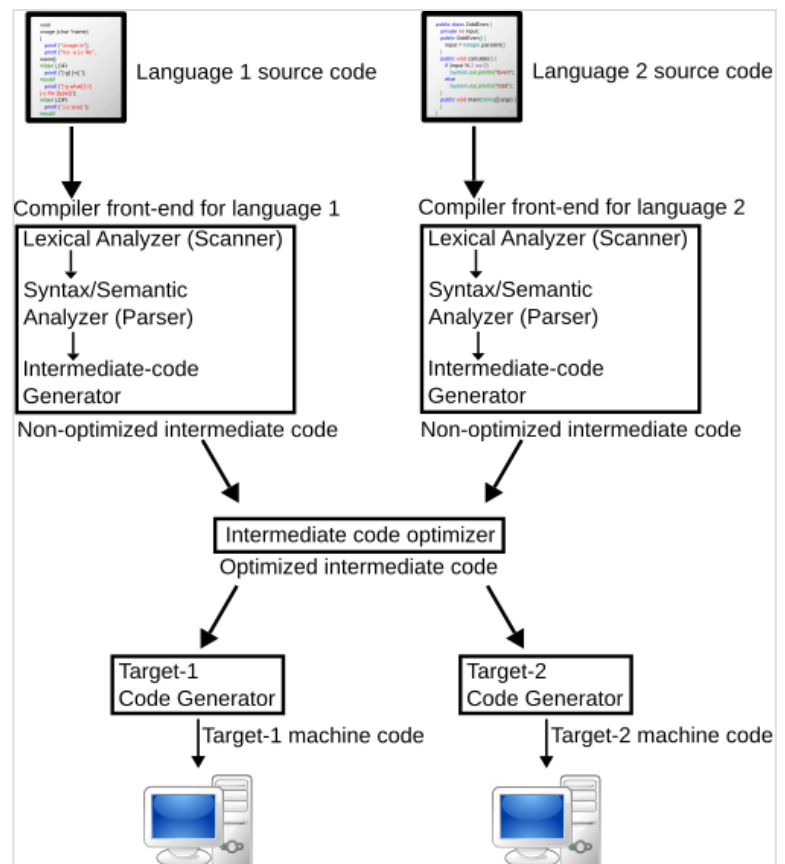
Theoretical computing concepts developed by scientists, mathematicians, and engineers formed the basis of digital modern computing development during World War II. Primitive binary languages evolved because digital devices only understand ones and zeros and the circuit patterns in the underlying machine architecture. In the late 1940s, assembly languages were created to offer a more workable abstraction of the computer architectures.^[5] Limited memory capacity of early computers led to substantial technical challenges when the first compilers were designed. Therefore, the compiling process needed to be divided into several small programs. The front end programs produce the analysis products used by the back end programs to generate target code. As computer technology provided more resources, compiler designs could align better with the compiling process.

It is usually more productive for a programmer to use a high-level language, so the development of high-level languages followed naturally from the abilities offered by digital computers. High-level languages are formal languages that are strictly defined by their syntax and semantics which form the high-level language architecture. Elements of these formal languages include:

- *Alphabet*, any finite set of symbols
- *String*, a finite sequence of symbols
- *Language*, any set of strings on an alphabet

The sentences in a language may be defined by a set of rules called a grammar.^[6]

Backus–Naur form (BNF) describes the syntax of "sentences" of a language. It was developed by John Backus and used for the syntax of Algol 60.^[7] The ideas derive from the context-free grammar concepts by linguist Noam Chomsky.^[8] "BNF and its extensions have become standard tools for describing the



A diagram of the operation of a typical multi-language, multi-target compiler

syntax of programming notations. In many cases, parts of compilers are generated automatically from a BNF description."^[9]

Between 1942 and 1945, Konrad Zuse designed the first (algorithmic) programming language for computers named Plankalkül ("Plan Calculus"). Zuse also envisioned a Planfertigungsgerät ("Plan assembly device") to automatically translate the mathematical formulation of a program into machine-readable punched film stock.^[10] While no implementation occurred until the 1970s, it presented concepts later seen in APL designed by Ken Iverson in the late 1950s.^[11] APL is a language for mathematical computations.

Between 1949 and 1951, Heinz Rutishauser proposed Superplan, a high-level language and automatic translator.^[12] His ideas were later refined by Friedrich L. Bauer and Klaus Samelson.^[13]

High-level language design during the formative years of digital computing provided useful programming tools for a variety of applications:

- FORTRAN (Formula Translation) for engineering and science applications is considered to be one of the first implemented high-level languages and first optimizing compiler.^[14]
- COBOL (Common Business-Oriented Language) evolved from A-0 and FLOW-MATIC to become the dominant high-level language for business applications.^[15]
- LISP (List Processor) for symbolic computation.^[16]

Compiler technology evolved from the need for a strictly defined transformation of the high-level source program into a low-level target program for the digital computer. The compiler could be viewed as a front end to deal with the analysis of the source code and a back end to synthesize the analysis into the target code. Optimization between the front end and back end could produce more efficient target code.^[17]

Some early milestones in the development of compiler technology:

- *May 1952:* Grace Hopper's team at Remington Rand wrote the compiler for the A-0 programming language (and coined the term *compiler* to describe it),^{[18][19][20]} although the A-0 compiler functioned more as a loader or linker than the modern notion of a full compiler.^{[21][22][23]}
- *1952, before September:* An Autocode compiler developed by Alick Glennie for the Manchester Mark I computer at the University of Manchester is considered by some to be the first compiled programming language.^[24]
- *1954–1957:* A team led by John Backus at IBM developed FORTRAN which is usually considered the first high-level language. In 1957, they completed a FORTRAN compiler that is generally credited as having introduced the first unambiguously complete compiler.^[25]
- *1959:* The Conference on Data Systems Language (CODASYL) initiated development of COBOL. The COBOL design drew on A-0 and FLOW-MATIC. By the early 1960s COBOL was compiled on multiple architectures.
- *1958–1960:* ALGOL 58 was the precursor to ALGOL 60. It introduced code blocks, a key advance in the rise of structured programming. ALGOL 60 was the first language to implement nested function definitions with lexical scope. It included recursion. Its syntax was defined using BNF. ALGOL 60 inspired many languages that followed it. Tony Hoare remarked: "... it was not only an improvement on its predecessors but also on nearly all its successors."^{[26][27]}
- *1958–1962:* John McCarthy at MIT designed LISP.^[28] The symbol processing abilities provided useful features for artificial intelligence research. In 1962, LISP 1.5 release noted

some tools: an interpreter written by Stephen Russell and Daniel J. Edwards, a compiler and assembler written by Tim Hart and Mike Levin.^[29]

Early operating systems and software were written in assembly language. In the 1960s and early 1970s, the use of high-level languages for system programming was still controversial due to resource limitations. However, several research and industry efforts began the shift toward high-level systems programming languages, for example, BCPL, BLISS, B, and C.

BCPL (Basic Combined Programming Language) designed in 1966 by Martin Richards at the University of Cambridge was originally developed as a compiler writing tool.^[30] Several compilers have been implemented, Richards' book provides insights to the language and its compiler.^[31] BCPL was not only an influential systems programming language that is still used in research^[32] but also provided a basis for the design of B and C languages.

BLISS (Basic Language for Implementation of System Software) was developed for a Digital Equipment Corporation (DEC) PDP-10 computer by W. A. Wulf's Carnegie Mellon University (CMU) research team. The CMU team went on to develop BLISS-11 compiler one year later in 1970.

Multics (Multiplexed Information and Computing Service), a time-sharing operating system project, involved MIT, Bell Labs, General Electric (later Honeywell) and was led by Fernando Corbató from MIT.^[33] Multics was written in the PL/I language developed by IBM and IBM User Group.^[34] IBM's goal was to satisfy business, scientific, and systems programming requirements. There were other languages that could have been considered but PL/I offered the most complete solution even though it had not been implemented.^[35] For the first few years of the Multics project, a subset of the language could be compiled to assembly language with the Early PL/I (EPL) compiler by Doug McIlory and Bob Morris from Bell Labs.^[36] EPL supported the project until a boot-strapping compiler for the full PL/I could be developed.^[37]

Bell Labs left the Multics project in 1969, and developed a system programming language B based on BCPL concepts, written by Dennis Ritchie and Ken Thompson. Ritchie created a boot-strapping compiler for B and wrote Unics (Uniplexed Information and Computing Service) operating system for a PDP-7 in B. Unics eventually became spelled Unix.

Bell Labs started the development and expansion of C based on B and BCPL. The BCPL compiler had been transported to Multics by Bell Labs and BCPL was a preferred language at Bell Labs.^[38] Initially, a front-end program to Bell Labs' B compiler was used while a C compiler was developed. In 1971, a new PDP-11 provided the resource to define extensions to B and rewrite the compiler. By 1973 the design of C language was essentially complete and the Unix kernel for a PDP-11 was rewritten in C. Steve Johnson started development of Portable C Compiler (PCC) to support retargeting of C compilers to new machines.^{[39][40]}

Object-oriented programming (OOP) offered some interesting possibilities for application development and maintenance. OOP concepts go further back but were part of LISP and Simula language science.^[41] Bell Labs became interested in OOP with the development of C++.^[42] C++ was first used in 1980 for systems programming. The initial design leveraged C language systems programming abilities with Simula concepts. Object-oriented facilities were added in 1983.^[43] The Cfront program implemented a C++ front-end for C84 language compiler. In subsequent years several C++ compilers were developed as C++ popularity grew.

In many application domains, the idea of using a higher-level language quickly caught on. Because of the expanding functionality supported by newer programming languages and the increasing complexity of computer architectures, compilers became more complex.

DARPA (Defense Advanced Research Projects Agency) sponsored a compiler project with Wulf's CMU research team in 1970. The Production Quality Compiler-Compiler PQCC design would produce a Production Quality Compiler (PQC) from formal definitions of source language and the target.^[44] PQCC tried to extend the term compiler-compiler beyond the traditional meaning as a parser generator (e.g., Yacc) without much success. PQCC might more properly be referred to as a compiler generator.

PQCC research into code generation process sought to build a truly automatic compiler-writing system. The effort discovered and designed the phase structure of the PQC. The BLISS-11 compiler provided the initial structure.^[45] The phases included analyses (front end), intermediate translation to virtual machine (middle end), and translation to the target (back end). TCOL was developed for the PQCC research to handle language specific constructs in the intermediate representation.^[46] Variations of TCOL supported various languages. The PQCC project investigated techniques of automated compiler construction. The design concepts proved useful in optimizing compilers and compilers for the (since 1995, object-oriented) programming language Ada.

The Ada *STONEMAN* document^[a] formalized the program support environment (APSE) along with the kernel (KAPSE) and minimal (MAPSE). An Ada interpreter NYU/ED supported development and standardization efforts with the American National Standards Institute (ANSI) and the International Standards Organization (ISO). Initial Ada compiler development by the U.S. Military Services included the compilers in a complete integrated design environment along the lines of the *STONEMAN* document. Army and Navy worked on the Ada Language System (ALS) project targeted to DEC/VAX architecture while the Air Force started on the Ada Integrated Environment (AIE) targeted to IBM 370 series. While the projects did not provide the desired results, they did contribute to the overall effort on Ada development.^[47]

Other Ada compiler efforts got underway in Britain at the University of York and in Germany at the University of Karlsruhe. In the U. S., Verdix (later acquired by Rational) delivered the Verdix Ada Development System (VADS) to the Army. VADS provided a set of development tools including a compiler. Unix/VADS could be hosted on a variety of Unix platforms such as DEC Ultrix and the Sun 3/60 Solaris targeted to Motorola 68020 in an Army CECOM evaluation.^[48] There were soon many Ada compilers available that passed the Ada Validation tests. The Free Software Foundation GNU project developed the GNU Compiler Collection (GCC) which provides a core capability to support multiple languages and targets. The Ada version GNAT is one of the most widely used Ada compilers. GNAT is free but there is also commercial support, for example, AdaCore, was founded in 1994 to provide commercial software solutions for Ada. GNAT Pro includes the GNU GCC based GNAT with a tool suite to provide an integrated development environment.

High-level languages continued to drive compiler research and development. Focus areas included optimization and automatic code generation. Trends in programming languages and development environments influenced compiler technology. More compilers became included in language distributions (PERL, Java Development Kit) and as a component of an IDE (VADS, Eclipse, Ada Pro). The interrelationship and interdependence of technologies grew. The advent of web services promoted growth of web languages and scripting languages. Scripts trace back to the early days of Command Line Interfaces (CLI) where the user could enter commands to be executed by the system. User Shell concepts

developed with languages to write shell programs. Early Windows designs offered a simple batch programming capability. The conventional transformation of these language used an interpreter. While not widely used, Bash and Batch compilers have been written. More recently sophisticated interpreted languages became part of the developers tool kit. Modern scripting languages include PHP, Python, Ruby and Lua. (Lua is widely used in game development.) All of these have interpreter and compiler support.^[49]

"When the field of compiling began in the late 50s, its focus was limited to the translation of high-level language programs into machine code ... The compiler field is increasingly intertwined with other disciplines including computer architecture, programming languages, formal methods, software engineering, and computer security."^[50] The "Compiler Research: The Next 50 Years" article noted the importance of object-oriented languages and Java. Security and parallel computing were cited among the future research targets.

Compiler construction

A compiler implements a formal transformation from a high-level source program to a low-level target program. Compiler design can define an end-to-end solution or tackle a defined subset that interfaces with other compiling tools e.g., preprocessors, assemblers, linkers. Design requirements include rigorously defined interfaces both internally between compiler components and externally between supporting toolsets.

In the early days, the approach taken to compiler design was directly affected by the complexity of the computer language to be processed, the experience of the person(s) designing it, and the resources available. Resource limitations led to the need to pass through the source code more than once.

A compiler for a relatively simple language written by one person might be a single, monolithic piece of software. However, as the source language grows in complexity the design may be split into a number of interdependent phases. Separate phases provide design improvements that focus development on the functions in the compiling process.

One-pass vs multi-pass compilers

Classifying compilers by number of passes has its background in the hardware resource limitations of computers. Compiling involves performing much work and early computers did not have enough memory to contain one program that did all of this work. As a result, compilers were split up into smaller programs which each made a pass over the source (or some representation of it) performing some of the required analysis and translations.

The ability to compile in a single pass has classically been seen as a benefit because it simplifies the job of writing a compiler and one-pass compilers generally compile source code faster than multi-pass compilers. Thus, partly driven by the resource limits of early systems, many early languages were specifically designed so that they could be compiled in one pass (e.g., Pascal).

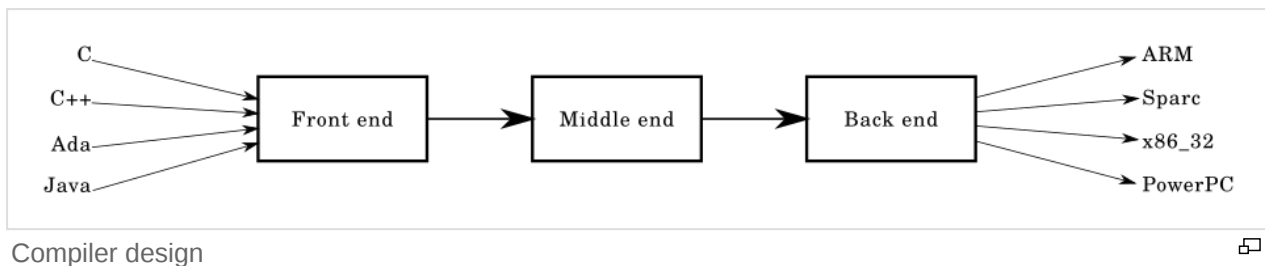
In some cases, the design of a language feature may require a compiler to perform more than one pass over the source. For example, consider a declaration appearing on line 20 of the source which affects the translation of a statement appearing on line 10. In this case, the first pass needs to gather information

about declarations appearing after statements that they affect, with the translation then performed during a subsequent pass.

The disadvantage of compiling in a single pass is that it is not possible to perform many of the sophisticated optimizations needed to generate high quality code. It can be difficult to count exactly how many passes an optimizing compiler makes. For instance, different phases of optimization may analyse one expression many times but only analyse another expression once.

Splitting a compiler up into small programs is a technique used by researchers interested in producing provably correct compilers. Proving the correctness of a set of small programs often requires less effort than proving the correctness of a larger, single, equivalent program.

Three-stage compiler structure



Regardless of the exact number of phases in the compiler design, the phases can be assigned to one of three stages. The stages include a front end, a middle end, and a back end.

- The *front end* scans the input and verifies syntax and semantics according to a specific source language. For statically typed languages it performs type checking by collecting type information. If the input program is syntactically incorrect or has a type error, it generates error and/or warning messages, usually identifying the location in the source code where the problem was detected; in some cases, the error may be (far) earlier in a program. Aspects of the front end include lexical analysis, syntax analysis, and semantic analysis. The front end transforms the input program into an intermediate representation (IR) for further processing by the middle end. This IR is usually a lower-level representation of the program with respect to the source code.
- The *middle end* performs optimizations on the IR that are independent of the CPU architecture being targeted. This source code/machine code independence is intended to enable generic optimizations to be shared between versions of the compiler supporting different languages and target processors. Examples of middle end optimizations are removal of useless (dead-code elimination) or unreachable code (reachability analysis), discovery and propagation of constant values (constant propagation), relocation of computation to a less frequently executed place (e.g., out of a loop), or specialization of computation based on the context, eventually producing the "optimized" IR that is used by the back end.
- The *back end* takes the optimized IR from the middle end. It may perform more analysis, transformations and optimizations that are specific for the target CPU architecture. The back end generates the target-dependent assembly code, performing register allocation in the process. The back end performs instruction scheduling, which re-orders instructions to keep parallel execution units busy by filling delay slots. Although most optimization problems are NP-hard, heuristic techniques for solving them are well-developed and implemented in production-quality compilers. Typically the output of a back end is machine code specialized for a particular processor and operating system.

This front/middle/back-end approach makes it possible to combine front ends for different languages with back ends for different CPUs while sharing the optimizations of the middle end.^[51] Practical examples of this approach are the GNU Compiler Collection, Clang (LLVM-based C/C++ compiler),^[52] and the Amsterdam Compiler Kit, which have multiple front-ends, shared optimizations and multiple back-ends.

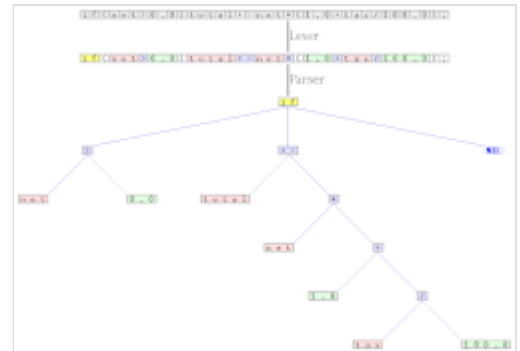
Front end

The front end analyzes the source code to build an internal representation of the program, called the intermediate representation (IR). It also manages the symbol table, a data structure mapping each symbol in the source code to associated information such as location, type and scope.

While the frontend can be a single monolithic function or program, as in a scannerless parser, it was traditionally implemented and analyzed as several phases, which may execute sequentially or concurrently. This method is favored due to its modularity and separation of concerns. Most commonly, the frontend is broken into three phases: lexical analysis (also known as lexing or scanning), syntax analysis (also known as parsing), and semantic analysis. Lexing and parsing comprise the syntactic analysis (word syntax and phrase syntax, respectively), and in simple cases, these modules (the lexer and parser) can be automatically generated from a grammar for the language, though in more complex cases these require manual modification. The lexical grammar and phrase grammar are usually context-free grammars, which simplifies analysis significantly, with context-sensitivity handled at the semantic analysis phase. The semantic analysis phase is generally more complex and written by hand, but can be partially or fully automated using attribute grammars. These phases themselves can be further broken down: lexing as scanning and evaluating, and parsing as building a concrete syntax tree (CST, parse tree) and then transforming it into an abstract syntax tree (AST, syntax tree). In some cases additional phases are used, notably *line reconstruction* and *preprocessing*, but these are rare.

The main phases of the front end include the following:

- *Line reconstruction* converts the input character sequence to a canonical form ready for the parser. Languages which stop their keywords or allow arbitrary spaces within identifiers require this phase. The top-down, recursive-descent, table-driven parsers used in the 1960s typically read the source one character at a time and did not require a separate tokenizing phase. Atlas Autocode and Imp (and some implementations of ALGOL and Coral 66) are examples of stopped languages whose compilers would have a *Line Reconstruction* phase.
- *Preprocessing* supports macro substitution and conditional compilation. Typically the preprocessing phase occurs before syntactic or semantic analysis; e.g., in the case of C, the preprocessor manipulates lexical tokens rather than syntactic forms. However, some languages such as Scheme support macro substitutions based on syntactic forms.



Lexer and parser example for C.

Starting from the sequence of characters "if(net>0.0)total+=net*(1.0+tax/100.0);", the scanner composes a sequence of tokens, and categorizes each of them, for example as identifier, reserved word, number literal, or operator. The latter sequence is transformed by the parser into a syntax tree, which is then treated by the remaining compiler phases. The scanner and parser handles the regular and properly context-free parts of the grammar for C, respectively.

- Lexical analysis (also known as *lexing* or *tokenization*) breaks the source code text into a sequence of small pieces called *lexical tokens*.^[53] This phase can be divided into two stages: the *scanning*, which segments the input text into syntactic units called *lexemes* and assigns them a category; and the *evaluating*, which converts lexemes into a processed value. A token is a pair consisting of a *token name* and an optional *token value*.^[54] Common token categories may include identifiers, keywords, separators, operators, literals and comments, although the set of token categories varies in different programming languages. The lexeme syntax is typically a regular language, so a finite-state automaton constructed from a regular expression can be used to recognize it. The software doing lexical analysis is called a lexical analyzer. This may not be a separate step—it can be combined with the parsing step in scannerless parsing, in which case parsing is done at the character level, not the token level.
- Syntax analysis (also known as *parsing*) involves parsing the token sequence to identify the syntactic structure of the program. This phase typically builds a parse tree, which replaces the linear sequence of tokens with a tree structure built according to the rules of a formal grammar which define the language's syntax. The parse tree is often analyzed, augmented, and transformed by later phases in the compiler.^[55]
- Semantic analysis adds semantic information to the parse tree and builds the symbol table. This phase performs semantic checks such as type checking (checking for type errors), or object binding (associating variable and function references with their definitions), or definite assignment (requiring all local variables to be initialized before use), rejecting incorrect programs or issuing warnings. Semantic analysis usually requires a complete parse tree, meaning that this phase logically follows the parsing phase, and logically precedes the code generation phase, though it is often possible to fold multiple phases into one pass over the code in a compiler implementation.

Middle end

The middle end, also known as *optimizer*, performs optimizations on the intermediate representation in order to improve the performance and the quality of the produced machine code.^[56] The middle end contains those optimizations that are independent of the CPU architecture being targeted.

The main phases of the middle end include the following:

- Analysis: This is the gathering of program information from the intermediate representation derived from the input; data-flow analysis is used to build use-define chains, together with dependence analysis, alias analysis, pointer analysis, escape analysis, etc. Accurate analysis is the basis for any compiler optimization. The control-flow graph of every compiled function and the call graph of the program are usually also built during the analysis phase.
- Optimization: the intermediate language representation is transformed into functionally equivalent but faster (or smaller) forms. Popular optimizations are inline expansion, dead-code elimination, constant propagation, loop transformation and even automatic parallelization.

Compiler analysis is the prerequisite for any compiler optimization, and they tightly work together. For example, dependence analysis is crucial for loop transformation.

The scope of compiler analysis and optimizations vary greatly; their scope may range from operating within a basic block, to whole procedures, or even the whole program. There is a trade-off between the granularity of the optimizations and the cost of compilation. For example, peephole optimizations are fast to perform during compilation but only affect a small local fragment of the code, and can be performed

independently of the context in which the code fragment appears. In contrast, interprocedural optimization requires more compilation time and memory space, but enable optimizations that are only possible by considering the behavior of multiple functions simultaneously.

Interprocedural analysis and optimizations are common in modern commercial compilers from HP, IBM, SGI, Intel, Microsoft, and Sun Microsystems. The free software GCC was criticized for a long time for lacking powerful interprocedural optimizations, but it is changing in this respect. Another open source compiler with full analysis and optimization infrastructure is Open64, which is used by many organizations for research and commercial purposes.

Due to the extra time and space needed for compiler analysis and optimizations, some compilers skip them by default. Users have to use compilation options to explicitly tell the compiler which optimizations should be enabled.

Back end

The back end is responsible for the CPU architecture specific optimizations and for code generation.^[56]

The main phases of the back end include the following:

- *Machine dependent optimizations*: optimizations that depend on the details of the CPU architecture that the compiler targets.^[57] A prominent example is peephole optimizations, which rewrites short sequences of assembler instructions into more efficient instructions.
- *Code generation*: the transformed intermediate language is translated into the output language, usually the native machine language of the system. This involves resource and storage decisions, such as deciding which variables to fit into registers and memory and the selection and scheduling of appropriate machine instructions along with their associated addressing modes (see also Sethi–Ullman algorithm). Debug data may also need to be generated to facilitate debugging.

Compiler correctness

Compiler correctness is the branch of software engineering that deals with trying to show that a compiler behaves according to its language specification.^[58] Techniques include developing the compiler using formal methods and using rigorous testing (often called compiler validation) on an existing compiler.

Compiled vis-à-vis interpreted languages

Higher-level programming languages usually appear with a type of translation in mind: either designed as compiled language or interpreted language. However, in practice there is rarely anything about a language that *requires* it to be exclusively compiled or exclusively interpreted, although it is possible to design languages that rely on re-interpretation at run time. The categorization usually reflects the most popular or widespread implementations of a language. For instance, BASIC is sometimes called an interpreted language, and C a compiled one, despite the existence of BASIC compilers and C interpreters.^[59]

Interpretation does not replace compilation completely. It only hides it from the user and makes it gradual. Even though an interpreter can itself be interpreted, a set of directly executed machine instructions is needed somewhere at the bottom of the execution stack (see [machine language](#)).

Furthermore, for optimization compilers can contain interpreter functionality, and interpreters may include ahead of time compilation techniques. For example, where an expression can be executed during compilation and the results inserted into the output program, then it prevents it having to be recalculated each time the program runs, which can greatly speed up the final program. Modern trends toward [just-in-time compilation](#) and [bytecode interpretation](#) at times blur the traditional categorizations of compilers and interpreters even further. [Meta-tracing](#) is an automated compiler synthesis approach which takes this further and can be used to synthesize a compiler from a language interpreter.

Some language specifications spell out that implementations *must* include a compilation facility; for example, [Common Lisp](#). However, there is nothing inherent in the definition of Common Lisp that stops it from being interpreted. Other languages have features that are very easy to implement in an interpreter, but make writing a compiler much harder; for example, [APL](#), [SNOBOL4](#),^[60] and many scripting languages allow programs to construct arbitrary source code at runtime with regular string operations, and then execute that code by passing it to a special [evaluation function](#). To implement these features in a compiled language, programs must usually be shipped with a [runtime library](#) that includes a version of the compiler itself.

Types

One classification of compilers is by the [platform](#) on which their generated code executes. This is known as the *target platform*.

A *native* or *hosted* compiler is one whose output is intended to directly run on the same type of computer and operating system that the compiler itself runs on. The output of a [cross compiler](#) is designed to run on a different platform. Cross compilers are often used when developing software for [embedded systems](#) that are not intended to support a software development environment.

The output of a compiler that produces code for a [virtual machine](#) (VM) may or may not be executed on the same platform as the compiler that produced it. For this reason, such compilers are not usually classified as native or cross compilers.

The lower level language that is the target of a compiler may itself be a [high-level programming language](#). C, viewed by some as a sort of portable assembly language, is frequently the target language of such compilers. For example, Cfront, the original compiler for C++, used C as its target language. The C code generated by such a compiler is usually not intended to be readable and maintained by humans, so [indent style](#) and creating pretty C intermediate code are ignored. Some of the features of C that make it a good target language include the [#line](#) directive, which can be generated by the compiler to support [debugging](#) of the original source, and the wide platform support available with C compilers.

While a common compiler type outputs machine code, there are many other types:

- [Source-to-source compilers](#) are a type of compiler that takes a high-level language as its input and outputs a high-level language. For example, an [automatic parallelizing compiler](#) will frequently take in a high-level language program as an input and then transform the

code and annotate it with parallel code annotations (e.g., [OpenMP](#)) or language constructs (e.g., Fortran's DOALL statements). Other terms for a source-to-source compiler are transcompiler or transpiler.^[61]

- [Bytecode compilers](#) compile to assembly language of a theoretical machine, like some [Prolog](#) implementations
 - This Prolog machine is also known as the [Warren Abstract Machine](#) (or WAM).
 - Bytecode compilers for [Java](#), [Python](#) are also examples of this category.
- [Just-in-time compilers](#) (JIT compiler) defer compiling until runtime. JIT compilers exist for many modern languages including Python, JavaScript, Smalltalk, Java, .NET's [Common Intermediate Language](#) (CIL), originally in Microsoft .NET Framework, and others. A JIT compiler generally runs inside an interpreter. When the interpreter detects that a code path is "hot", meaning it is executed frequently, the JIT compiler will be invoked and compile the "hot" code for increased performance.
 - For some languages, such as Java, applications are first compiled using a bytecode compiler and delivered in a machine-independent intermediate representation. A bytecode interpreter executes the bytecode, but the JIT compiler will translate the bytecode to machine code when increased performance is necessary.^[62]
- [Hardware compilers](#) (also known as synthesis tools) are compilers which input is a hardware description language and which output is a description, in the form of a [netlist](#) or otherwise, of a hardware configuration.
 - The output of these compilers target computer hardware at a very low level, for example a [field-programmable gate array](#) (FPGA) or structured [application-specific integrated circuit](#) (ASIC).^[63] Such compilers are said to be hardware compilers, because the source code they compile effectively controls the final configuration of the hardware and how it operates. The output of the compilation is only an interconnection of [transistors](#) or [lookup tables](#).
 - An example of hardware compiler is XST, the Xilinx Synthesis Tool used for configuring FPGAs.^[64] Similar tools are available from Altera,^[65] Synplicity, Synopsys and other hardware vendors.^[66]
 - Research systems compile subsets of high level serial languages, such as Python or C++, directly into parallelized digital logic. This is typically easier to do for functional languages or functional subsets of multi-paradigm languages.^[67]
- A program that translates from a low-level language to a higher level one is a [decompiler](#).^[68]
- A program that translates into an object code format that is unsupported on the compiling machine is called a [cross compiler](#) and is commonly used to prepare code for execution on embedded software applications.^[69]
- A program that rewrites object code back into the same type of object code while applying optimisations and transformations is a [binary recompiler](#).

Assemblers, which translate human readable [assembly language](#) to the [machine code](#) instructions executed by hardware, are not considered compilers.^{[70][b]} (The inverse program that translates machine code to assembly language is called a [disassembler](#).)

See also



**Computer
programming portal**

- [Abstract interpretation](#)
- [Assembler](#)
- [Bottom-up parsing](#)
- [Compile and go system](#)
- [Compile farm](#)
- [List of compiler books](#)
- [List of compilers](#)
- [Metacompilation](#)
- [Program transformation](#)
- [Software build](#)

Notes and references

- a. [United States Department of Defense \(18 February 1980\) Stoneman requirements \(https://en.wikisource.org/wiki/Stoneman_requirements\)](https://en.wikisource.org/wiki/Stoneman_requirements)
- b. "The many source-language features described in the preceding section result in a number of salient differences between compilers and assemblers. On any one item the distinction may not be clear-cut. Moreover, it may be difficult to distinguish a simple compiler from a powerful macro assembler. Nevertheless, the differences are usually substantial enough that there remains a qualitative distinction between assemblers and compilers."
1. "Encyclopedia: Definition of Compiler" (<https://www.pcmag.com/encyclopedia/term/compiler>). *PCMag.com*. Retrieved 2 July 2022.
2. Aho, Alfred V.; Sethi, Ravi; Ullman, Jeffrey D. (2007). *Compilers: Principles, Techniques, and Tools* (2nd ed.).
3. Sudarsanam, Ashok; Malik, Sharad; Fujita, Masahiro (2002). "A Retargetable Compilation Methodology for Embedded Digital Signal Processors Using a Machine-Dependent Code Optimization Library". *Readings in Hardware/Software Co-Design*. Elsevier. pp. 506–515. doi:10.1016/b978-155860702-6/50045-4 (<https://doi.org/10.1016%2Fb978-155860702-6%2F50045-4>). ISBN 9781558607026. "A compiler is a computer program that translates a program written in a high-level language (HLL), such as C, into an equivalent assembly language program [2]."
4. Sun, Chengnian; Le, Vu; Zhang, Qirun; Su, Zhendong (2016). "Toward understanding compiler bugs in GCC and LLVM" (<http://dl.acm.org/citation.cfm?doid=2931037.2931074>). *Proceedings of the 25th International Symposium on Software Testing and Analysis*. ISSTA 2016. ACM. pp. 294–305. doi:10.1145/2931037.2931074 (<https://doi.org/10.1145%2F2931037.2931074>). ISBN 9781450343909. S2CID 8339241 (<https://api.semanticscholar.org/CorpusID:8339241>).
5. Baghai, Christian (4 April 2023). "The Evolution of Programming Languages: From Primitive Binary to High-Level Abstractions" (<https://christianbaghai.medium.com/the-evolution-of-programming-languages-from-primitive-binary-to-high-level-abstractions-7b8e4b7a2521>). *Medium*. Retrieved 10 July 2024.
6. Lecture notes. Compilers: Principles, Techniques, and Tools. Jing-Shin Chang. Department of Computer Science & Information Engineering. National Chi-Nan University
7. Naur, P. et al. "Report on ALGOL 60". *Communications of the ACM* 3 (May 1960), 299–314.
8. Chomsky, Noam; Lightfoot, David W. (2002). *Syntactic Structures*. Walter de Gruyter. ISBN 978-3-11-017279-9.
9. Gries, David (2012). "Appendix 1: Backus-Naur Form" (<https://books.google.com/books?id=QFrlBwAAQBAJ&q=1461259835&pg=PA304>). *The Science of Programming*. Springer Science & Business Media. p. 304. ISBN 978-1461259831.

10. Hellige, Hans Dieter, ed. (2004) [November 2002]. Written at Bremen, Germany. *Geschichten der Informatik - Visionen, Paradigmen, Leit motive* (in German) (1 ed.). Berlin / Heidelberg, Germany: Springer-Verlag. pp. 45, 104, 105. doi:10.1007/978-3-642-18631-8 (<https://doi.org/10.1007%2F978-3-642-18631-8>). ISBN 978-3-540-00217-8. ISBN 3-540-00217-0. (xii+514 pages)
11. Iverson, Kenneth E. (1962). *A Programming Language* (https://archive.org/details/programmenglangu00iver_0). John Wiley and Sons. ISBN 978-0-471430-14-8.
12. Rutishauser, Heinz (1951). "Über automatische Rechenplanfertigung bei programmgesteuerten Rechenanlagen". *Zeitschrift für Angewandte Mathematik und Mechanik* (in German). **31**: 255. doi:10.1002/zamm.19510310820 (<https://doi.org/10.1002%2Fzamm.19510310820>).
13. Fothe, Michael; Wilke, Thomas, eds. (2015) [2014-11-14]. Written at Jena, Germany. *Keller, Stack und automatisches Gedächtnis – eine Struktur mit Potenzial* (<https://dl.gi.de/bitstream/handle/20.500.12116/4381/lni-t-7.pdf?sequence=1&isAllowed=y>) [*Cellar, stack and automatic memory - a structure with potential*] (PDF) (Tagungsband zum Kolloquium 14. November 2014 in Jena). GI Series: Lecture Notes in Informatics (LNI) – Thematics (in German). Vol. T-7. Bonn, Germany: Gesellschaft für Informatik (GI) / Köllen Druck + Verlag GmbH. pp. 20–21. ISBN 978-3-88579-426-4. ISSN 1614-3213 (<https://search.worldcat.org/issn/1614-3213>). Archived (<https://web.archive.org/web/20200412122706/https://dl.gi.de/bitstream/handle/20.500.12116/4381/lni-t-7.pdf?sequence=1&isAllowed=y>) (PDF) from the original on 12 April 2020. Retrieved 12 April 2020. [1] (77 pages)
14. Backus, John. "The history of FORTRAN I, II and III" (<http://www.softwarepreservation.org/projects/FORTRAN/paper/p25-backus.pdf>) (PDF). *History of Programming Languages*. Archived (<https://ghostarchive.org/archive/20221010/http://www.softwarepreservation.org/projects/FORTRAN/paper/p25-backus.pdf>) (PDF) from the original on 10 October 2022 – via Softwarepreservation.org.
15. Porter Adams, Vicki (5 October 1981). "Captain Grace M. Hopper: the Mother of COBOL". *InfoWorld*. **3** (20): 33. ISSN 0199-6649.
16. McCarthy, J.; Brayton, R.; Edwards, D.; Fox, P.; Hodes, L.; Luckham, D.; Maling, K.; Park, D.; Russell, S. (March 1960). "LISP I Programmers Manual" (PDF). Boston, Massachusetts: Artificial Intelligence Group, M.I.T. Computation Center and Research Laboratory.
17. Compilers Principles, Techniques, & Tools 2nd edition by Aho, Lam, Sethi, Ullman ISBN 0-321-48681-1
18. Hopper, Grace Murray (1952). "The education of a computer". *Proceedings of the 1952 ACM national meeting (Pittsburgh) on - ACM '52*. pp. 243–249. doi:10.1145/609784.609818 (<https://doi.org/10.1145%2F609784.609818>). S2CID 10081016 (<https://api.semanticscholar.org/CorpusID:10081016>).
19. Ridgway, Richard K. (1952). "Compiling routines". *Proceedings of the 1952 ACM national meeting (Toronto) on - ACM '52*. pp. 1–5. doi:10.1145/800259.808980 (<https://doi.org/10.1145%2F800259.808980>). S2CID 14878552 (<https://api.semanticscholar.org/CorpusID:14878552>).
20. "List of early compilers and assemblers" (<http://shape-of-code.coding-guidelines.com/2017/05/21/evidence-for-28-possible-compilers-in-1957>).
21. Hopper, Grace. "Keynote Address" (<https://dl.acm.org/doi/pdf/10.1145/800025.1198341>). *Proceedings of the ACM SIGPLAN History of Programming Languages (HOPL) conference, June 1978*. doi:10.1145/800025.1198341 (<https://doi.org/10.1145%2F800025.1198341>).
22. Bruderer, Herbert (21 December 2022). "Did Grace Hopper Create the First Compiler?" (<https://cacm.acm.org/blogs/blog-cacm/268001-did-grace-hopper-create-the-first-compiler/fulltext>).

23. Strawn, George; Strawn, Candace (2015). "Grace Hopper: Compilers and Cobol" (<https://www.computer.org/csdl/magazine/it/2015/01/mit2015010062/13rUxCitFF>). *IT Professional*. **17** (Jan.-Feb. 2015): 62–64. Bibcode:2015ITPro..17a..62S (<https://ui.adsabs.harvard.edu/abs/2015ITPro..17a..62S>). doi:10.1109/MITP.2015.6 (<https://doi.org/10.1109%2FMITP.2015.6>).
24. Knuth, Donald E.; Pardo, Luis Trabb, "Early development of programming languages", *Encyclopedia of Computer Science and Technology* (Marcel Dekker) 7: 419–493
25. Backus, John (1 June 1978), "The history of Fortran I, II, and III" (<https://dl.acm.org/doi/10.1145/800025.1198345>), *History of programming languages*, New York, NY, USA: Association for Computing Machinery, pp. 25–74, doi:10.1145/800025.1198345 (<https://doi.org/10.1145%2F800025.1198345>), ISBN 978-0-12-745040-7, retrieved 9 October 2024
26. Hoare, C.A.R. (December 1973). "Hints on Programming Language Design" (http://www.eecs.umich.edu/~bchandra/courses/papers/Hoare_Hints.pdf) (PDF). p. 27. Archived (https://ghostarchive.org/archive/20221010/http://www.eecs.umich.edu/~bchandra/courses/papers/Hoare_Hints.pdf) (PDF) from the original on 10 October 2022. (This statement is sometimes erroneously attributed to Edsger W. Dijkstra, also involved in implementing the first ALGOL 60 compiler.)
27. Abelson, Hal; Dybvig, R. K.; et al. Rees, Jonathan; Clinger, William (eds.). "Revised(3) Report on the Algorithmic Language Scheme, (Dedicated to the Memory of ALGOL 60)" (http://groups.csail.mit.edu/mac/ftpdir/scheme-reports/r3rs-html/r3rs_toc.html). Retrieved 20 October 2009.
28. "Recursive Functions of Symbolic Expressions and Their Computation by Machine (<https://dspace.mit.edu/bitstream/handle/1721.1/6096/AIM-008.pdf?sequence=2>)", *Communications of the ACM*, April 1960
29. McCarthy, John; Abrahams, Paul W.; Edwards, Daniel J.; Hart, Timothy P.; Levin, Michael I. (1965). *Lisp 1.5 Programmers Manual* (<https://books.google.com/books?id=68j6lEJjMQwC&pg=PR1>). The MIT Press. ISBN 978-0-26213011-0.
30. "BCPL: A tool for compiler writing and system programming (http://prog.vub.ac.be/~tjdhondt/ESL/BCPL_to_Cfront_files/p557-richards.pdf)" M. Richards, University Mathematical Laboratory Cambridge, England 1969
31. BCPL: The Language and Its Compiler, M Richards, Cambridge University Press (first published 31 December 1981)
32. The BCPL Cintsys and Cintpos User Guide, M. Richards, 2017
33. Corbató, F. J.; Vyssotsky, V. A. "Introduction and Overview of the MULTICS System" (<https://multicians.org/fjcc1.html>). *1965 Fall Joint Computer Conference*. Multicians.org.
34. Report II of the SHARE Advanced Language Development Committee, 25 June 1964
35. Multicians.org "The Choice of PL/I" article, Editor /tom Van Vleck
36. "PL/I As a Tool for System Programming", F.J. Corbato, *Datamation* 6 May 1969 issue
37. "The Multics PL/1 Compiler (<https://www.computer.org/csdl/proceedings/afips/1969/5074/00/50740187.pdf>)", R. A. Freiburghouse, GE, Fall Joint Computer Conference 1969
38. Dennis M. Ritchie, "The Development of the C Language (<https://www.bell-labs.com/usr/dmr/www/chist.pdf>)", *ACM Second History of Programming Languages Conference*, April 1993
39. S.C. Johnson, "a Portable C Compiler: Theory and Practice", 5th ACM POPL Symposium, January 1978
40. A. Snyder, A Portable Compiler for the Language C (<https://apps.dtic.mil/sti/pdfs/ADA010218.pdf>), MIT, 1974.
41. K. Nygaard, University of Oslo, Norway, "Basic Concepts in Object Oriented Programming (<http://www.cs.kent.edu/~durand/CS43101Fall2004/resources/BasicConceptsOOP-Nygaard1986.pdf>)", *SIGPLAN Notices* V21, 1986
42. B. Stroustrup: "What is Object-Oriented Programming?" *Proceedings 14th ASU Conference*, 1986.

43. Bjarne Stroustrup, "An Overview of the C++ Programming Language", Handbook of Object Technology (Editor: Saba Zamir, ISBN 0-8493-3135-8)
44. Leverett, Cattell, Hobbs, Newcomer, Reiner, Schatz, Wulf: "An Overview of the Production Quality Compiler-Compiler Project", CMU-CS-89-105, 1979
45. W. Wulf, K. Nori, "Delayed binding in PQCC generated compilers (<https://apps.dtic.mil/sti/pdfs/ADA125935.pdf>)", CMU Research Showcase Report, CMU-CS-82-138, 1982
46. Joseph M. Newcomer, David Alex Lamb, Bruce W. Leverett, Michael Tighe, William A. Wulf - Carnegie-Mellon University and David Levine, Andrew H. Reinerit - Intermetrics: "TCOL Ada: Revised Report on An Intermediate Representation for the DOD Standard Programming Language", 1979
47. William A. Whitaker, "Ada - the project: the DoD High Order Working Group", ACM SIGPLAN Notices (Volume 28, No. 3, March 1991)
48. CECOM Center for Software Engineering Advanced Software Technology, "Final Report - Evaluation of the ACEC Benchmark Suite for Real-Time Applications", AD-A231 968, 1990
49. P. Biggar, E. de Vries, D. Gregg, "A Practical Solution for Scripting Language Compilers", submission to Science of Computer Programming, 2009
50. M. Hall, D. Padua, K. Pingali, "Compiler Research: The Next 50 Years", ACM Communications 2009 Vol 54 #2
51. Cooper and Torczon 2012, p. 8
52. Lattner, Chris (2017). "LLVM" (<http://www.aosabook.org/en/llvm.html>). In Brown, Amy; Wilson, Greg (eds.). *The Architecture of Open Source Applications*. Archived (<https://web.archive.org/web/20161202070941/http://aosabook.org/en/llvm.html>) from the original on 2 December 2016. Retrieved 28 February 2017.
53. Aho, Lam, Sethi, Ullman 2007, p. 5-6, 109-189
54. Aho, Lam, Sethi, Ullman 2007, p. 111
55. Aho, Lam, Sethi, Ullman 2007, p. 8, 191-300
56. Blindell, Gabriel Hjort (3 June 2016). *Instruction selection: Principles, methods, and applications*. Switzerland: Springer. ISBN 978-3-31934019-7. OCLC 951745657 (<https://search.worldcat.org/oclc/951745657>).
57. Cooper and Torczon (2012), p. 540
58. "S1-A Simple Compiler" (<https://dx.doi.org/10.1002/9781118112762.ch12>), *Compiler Construction Using Java, JavaCC, and Yacc*, Hoboken, NJ, US: John Wiley and Sons, 28 February 2012, pp. 289–329, doi:10.1002/9781118112762.ch12 (<https://doi.org/10.1002/9781118112762.ch12>), ISBN 978-1-118-11276-2, retrieved 17 May 2023
59. "Compiler vs. Interpreter in Programming" (<https://builtin.com/software-engineering-perspectives/compiler-vs-interpreter>). *Built In*. Retrieved 25 May 2025.
60. "SNOBOL4 Programming Language" (<https://sourceforge.net/projects/snobol4/>). *SourceForge*. 27 October 2023. Retrieved 25 May 2025.
61. Ilyushin, Evgeniy; Namiot, Dmitry (2016). "On source-to-source compilers" (<https://web.archive.org/web/20220913223759/https://cyberleninka.ru/article/n/on-source-to-source-compilers/pdf>). *International Journal of Open Information Technologies*. **4** (5): 48–51. Archived from the original (<https://cyberleninka.ru/article/n/on-source-to-source-compilers/pdf>) on 13 September 2022. Retrieved 14 September 2022.
62. Aycock, John (2003). "A Brief History of Just-in-Time". *ACM Comput. Surv.* **35** (2): 93–113. doi:10.1145/857076.857077 (<https://doi.org/10.1145/857076.857077>). S2CID 15345671 (<https://api.semanticscholar.org/CorpusID:15345671>).

63. Swartz, Jordan S.; Betz, Vaughn; Rose, Jonathan (22–25 February 1998). "A fast routability-driven router for FPGAs" (<http://www.eecg.toronto.edu/~vaughn/papers/fpga98.pdf>) (PDF). *Proceedings of the 1998 ACM/SIGDA sixth international symposium on Field programmable gate arrays - FPGA '98*. Monterey, CA: ACM. pp. 140–149. doi:10.1145/275107.275134 (<https://doi.org/10.1145%2F275107.275134>). ISBN 978-0897919784. S2CID 7128364 (<https://api.semanticscholar.org/CorpusID:7128364>). Archived (<https://web.archive.org/web/20170809012611/http://www.eecg.toronto.edu/~vaughn/papers/fpga98.pdf>) (PDF) from the original on 9 August 2017.
64. Xilinx Staff (2009). "XST Synthesis Overview" (http://www.xilinx.com/support/documentation/sw_manuals/xilinx11/ise_c_using_xst_for_synthesis.htm). Xilinx, Inc. Archived (https://web.archive.org/web/20161102004019/http://www.xilinx.com/support/documentation/sw_manuals/xilinx11/ise_c_using_xst_for_synthesis.htm) from the original on 2 November 2016. Retrieved 28 February 2017.
65. Altera Staff (2017). "Spectra-Q™ Engine" (<https://web.archive.org/web/20161010221724/http://www.altera.com/products/design-software/fpga-design/quartus-prime/features/spectra-q.html>). Altera.com. Archived from the original (<https://www.altera.com/products/design-software/fpga-design/quartus-prime/features/spectra-q.html>) on 10 October 2016. Retrieved 28 February 2017.
66. Nigam, Rachit; Thomas, Samuel; Li, Zhijing; Sampson, Adrian (April 2021). "A compiler infrastructure for accelerator generators" (<https://dl.acm.org/doi/10.1145/3445814.3446712>). *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. pp. 804–817. arXiv:2102.09713 (<https://arxiv.org/abs/2102.09713>). doi:10.1145/3445814.3446712 (<https://doi.org/10.1145%2F3445814.3446712>). ISBN 978-1-4503-8317-2. Retrieved 20 May 2026.
67. Jurkans, K; Fox, C (2023). *Python Subset to Digital Logic Dataflow Compiler for Robots and IoT*. IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom-2023).
68. Metula, Erez (2011). "Tools of the Trade". *Managed Code Rootkits*. pp. 39–62. doi:10.1016/B978-1-59749-574-5.00003-9 (<https://doi.org/10.1016%2FB978-1-59749-574-5.00003-9>). ISBN 978-1-59749-574-5.
69. Chandrasekaran, Siddharth (26 January 2018). "Cross Compilation Demystified" (<https://embedjournal.com/cross-compilation-demystified/>). *embedjournal.com*. Retrieved 5 March 2023.
70. Calingaert and Horowitz 1979, pp. 186-187

Further reading

- Aho, Alfred V.; Sethi, Ravi; Ullman, Jeffrey D. (1986). *Compilers: Principles, Techniques, and Tools* (1st ed.). Addison-Wesley. ISBN 9780201100884.
- Allen, Frances E. (September 1981). "A History of Language Processor Technology in IBM". *IBM Journal of Research and Development*. **25** (5). IBM: 535–548. doi:10.1147/rd.255.0535 (<https://doi.org/10.1147%2Frd.255.0535>).
- Allen, Randy; Kennedy, Ken (2001). *Optimizing Compilers for Modern Architectures*. Morgan Kaufmann Publishers. ISBN 978-1-55860-286-1.
- Appel, Andrew Wilson (2002). *Modern Compiler Implementation in Java* (2nd ed.). Cambridge University Press. ISBN 978-0-521-82060-8.
- Appel, Andrew Wilson (1998). *Modern Compiler Implementation in ML* (<https://books.google.com/books?id=8APOYafUt-oC>). Cambridge University Press. ISBN 978-0-521-58274-2.
- Bornat, Richard (1979). *Understanding and Writing Compilers: A Do It Yourself Guide* (http://web.archive.org/web/20070615132948/http://www.cs.mdx.ac.uk/staffpages/r_bornat/books/compiling.pdf) (PDF). Macmillan Publishing. ISBN 978-0-333-21732-0. Archived from the

original (http://www.cs.mdx.ac.uk/staffpages/r_bornat/books/compiling.pdf) (PDF) on 15 June 2007. Retrieved 11 April 2007.

- Calingaert, Peter (1979). Horowitz, Ellis (ed.). *Assemblers, Compilers, and Program Translation* (<https://archive.org/details/assemblerscompil00cali>). Computer software engineering series (1st printing, 1st ed.). Potomac, Maryland: Computer Science Press, Inc. ISBN 0-914894-23-4. ISSN 0888-2088 (<https://search.worldcat.org/issn/0888-2088>). LCCN 78-21905 (<https://lccn.loc.gov/78-21905>). Retrieved 20 March 2020. (2+xiv+270+6 pages)
- Cooper, Keith Daniel; Torczon, Linda (2012). *Engineering a compiler* (2nd ed.). Amsterdam, Netherlands: Elsevier/Morgan Kaufmann. p. 8. ISBN 978-0-12088478-0. OCLC 714113472 (<https://search.worldcat.org/oclc/714113472>).
- Gries, David (1971). *Compiler Construction for Digital Computers* (in English, Spanish, Japanese, Chinese, Italian, and Russian). New York: John Wiley and Sons. ISBN 0-471-32776-X. "The first text on compiler construction."
- McKeeman, William Marshall; Horning, James J.; Wortman, David B. (1970). *A Compiler Generator* (<https://archive.org/details/compilergenerato00mcke>). Englewood Cliffs, New Jersey: Prentice-Hall. ISBN 978-0-13-155077-3.
- Muchnick, Steven (1997). *Advanced Compiler Design and Implementation* (<https://archive.org/details/advancedcompiler00much>). Morgan Kaufmann Publishers. ISBN 978-1-55860-320-2.
- Scott, Michael Lee (2005). *Programming Language Pragmatics* (<https://books.google.com/books?id=4LMtA2wOsPcC>) (2nd ed.). Morgan Kaufmann. ISBN 978-0-12-633951-2.
- Srikant, Y. N.; Shankar, Priti (2003). *The Compiler Design Handbook: Optimizations and Machine Code Generation* (https://books.google.com/books?id=0K_jlsgyNpoC). CRC Press. ISBN 978-0-8493-1240-3.
- Terry, Patrick D. (1997). *Compilers and Compiler Generators: An Introduction with C++* (<http://scifac.ru.ac.za/compilers/conts.htm>). International Thomson Computer Press. ISBN 978-1-85032-298-6.
- Wirth, Niklaus (1996). *Compiler Construction* (<https://web.archive.org/web/20170217071020/http://www.ethoberon.ethz.ch/WirthPubl/CBEAll.pdf>) (PDF). Addison-Wesley. ISBN 978-0-201-40353-4. Archived from the original (<http://www.ethoberon.ethz.ch/WirthPubl/CBEAll.pdf>) (PDF) on 17 February 2017. Retrieved 24 April 2012.
- LLVM community. "The LLVM Target-Independent Code Generator" (<http://llvm.org/docs/CodeGenerator.html#built-in-register-allocators>). *LLVM Documentation*. Retrieved 17 June 2016.
- Compiler textbook references (<https://web.archive.org/web/20150103161301/http://www.informatik.uni-trier.de/~ley/db/books/compiler/index.html>) A collection of references to mainstream Compiler Construction Textbooks

External links

- Incremental Approach to Compiler Construction (<http://scheme2006.cs.uchicago.edu/11-ghouloum.pdf>) – a PDF tutorial
- Basics of Compiler Design (<https://web.archive.org/web/20180515111448/http://www.diku.dk:80/hjemmesider/ansatte/torbenm/Basics/>) at the [Wayback Machine](#) (archived 15 May 2018)
- Short animation (https://www.youtube.com/watch?v=_C5AHaS1mOA) on YouTube explaining the key conceptual difference between compilers and interpreters
- Syntax Analysis & LL1 Parsing (<https://www.youtube.com/watch?v=QPCC2sbukeo>) on YouTube

- [Let's Build a Compiler \(http://compilers.iecc.com/crenshaw/\)](http://compilers.iecc.com/crenshaw/), by Jack Crenshaw
 - [Forum about compiler development \(https://web.archive.org/web/20141010102940/http://www.compdev.net/\)](https://web.archive.org/web/20141010102940/http://www.compdev.net/) at the [Wayback Machine](#) (archived 10 October 2014)
-

Retrieved from "<https://en.wikipedia.org/w/index.php?title=Compiler&oldid=1361469206>"